

XML

Explainable Machine Learning

Project Progress

Explainable Reinforcement Learning for Autonomous Vehicles

Ghanendra Singh, AI, IITH, 2024

Problem

Interpretable Machine Learning: Fundamental Principles and 10 Grand Challenges
[Rudin et al., 2021]

Challenge 10: Reinforcement Learning

“As far as we know, there are currently no **general, interpretable, well performing** methods for deep reinforcement learning that allow transparency in the agent’s actions or intent.”

Overview: Agent Model

Agent consists of three components that work closely together: **Vision (V)**, **Memory (M)**, and **Controller (C)**.

1. Agent has a **visual sensory** component that **compresses** what it sees into a small latent representation.
2. It also has a **memory** component that makes predictions about future codes based on historical information.
3. **Decision-making** component that decides what actions to take based only on the representations created by its vision and memory components.

At each time step, our agent receives an **observation** from the environment.

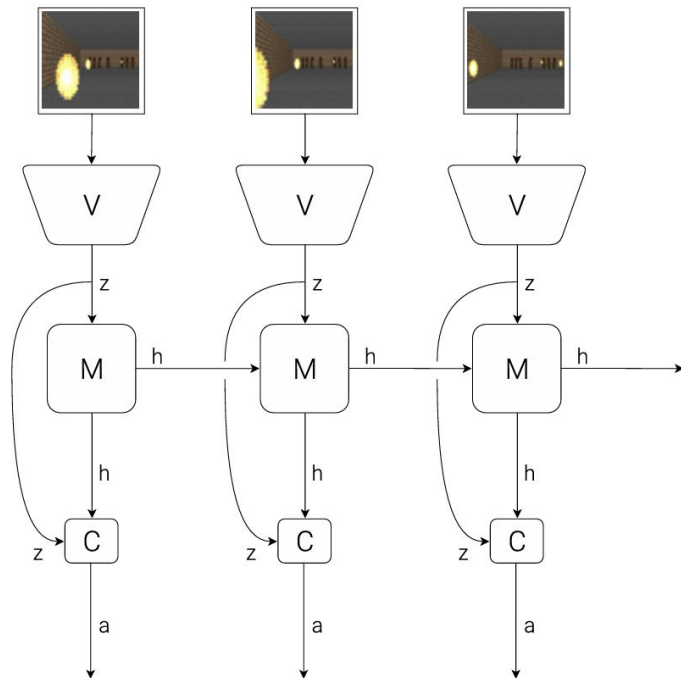
World Model

The **Vision Model (V)** encodes the high-dimensional observation into a low-dimensional latent vector.

The **Memory RNN (M)** integrates the historical codes to create a representation that can predict future states.

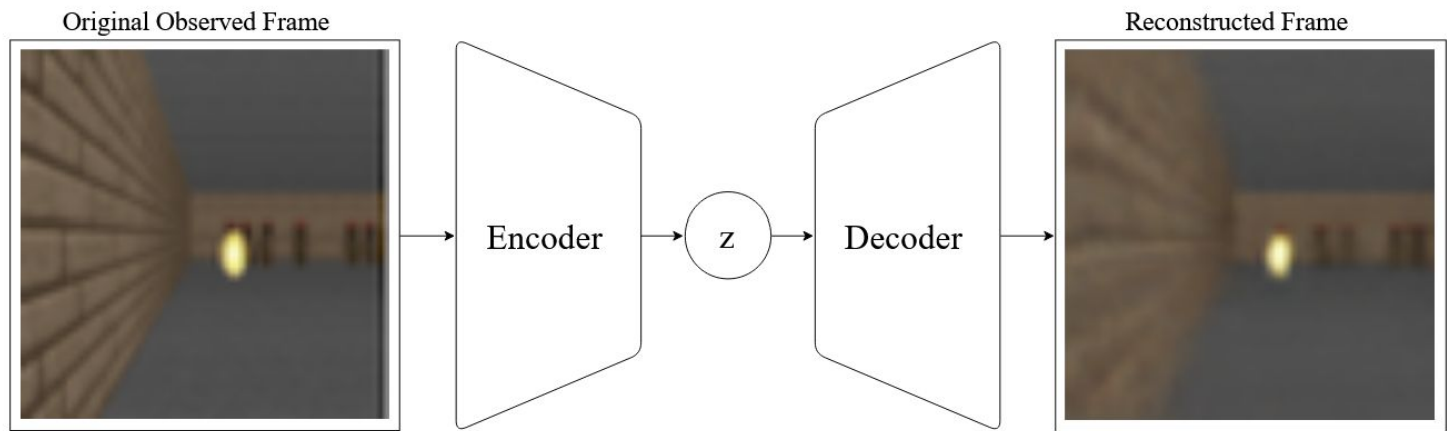
A small **Controller (C)** uses the representations from both **V** and **M** to select good actions.

The agent performs **actions** that go back and affect the environment.



Recap: VAE (V) Model

The environment provides our agent with a high dimensional input observation at each time step. This input is usually a 2D image frame that is part of a video sequence. The role of the V model is to learn an abstract, compressed representation of each observed input frame.



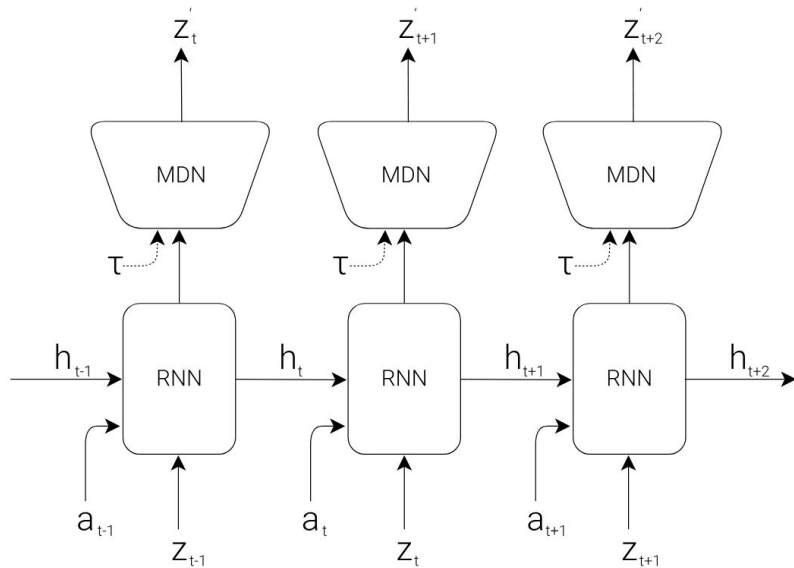
Recap: MDN-RNN (M) Model

Role of the V model to compress what the agent sees at each time frame.

Now, to compress what happens over time.
Role of the M model is to predict the future.

The M model serves as a predictive model of the future z vectors that V is expected to produce.

Because many complex environments are stochastic in nature, we train our RNN to output a probability density function $p(z)$ instead of a deterministic prediction of z .



RNN with a Mixture Density Network output layer. The MDN outputs the parameters of a mixture of Gaussian distribution used to sample a prediction of the next latent vector z .

a_t is the action taken at time t and h_t is the *hidden state* of the RNN at time t .

Recap: Controller (C) Model

The Controller (C) model is responsible for determining the **course of actions** to take in order to maximize the expected cumulative reward of the agent during a rollout of the environment.

In the experiments, C is as simple and small as possible, and trained separately from V and M, so that most of our agent's complexity resides in the world model (V and M).

C is a simple single layer linear model that maps **zt** and **ht** directly to action **at** at each time step:

$$\mathbf{a_t} = \mathbf{W_c}[\mathbf{z_t} \ \mathbf{h_t}] + \mathbf{b_c}$$ (This can be a complex non-linear function for motion planning)

In this linear model, $\mathbf{W_c}$ and $\mathbf{b_c}$ are the weight matrix and bias vector that maps the concatenated **input vector** **$[\mathbf{Z_t} \ \mathbf{H_t}]$** to the output action vector **$\mathbf{a_t}$** .

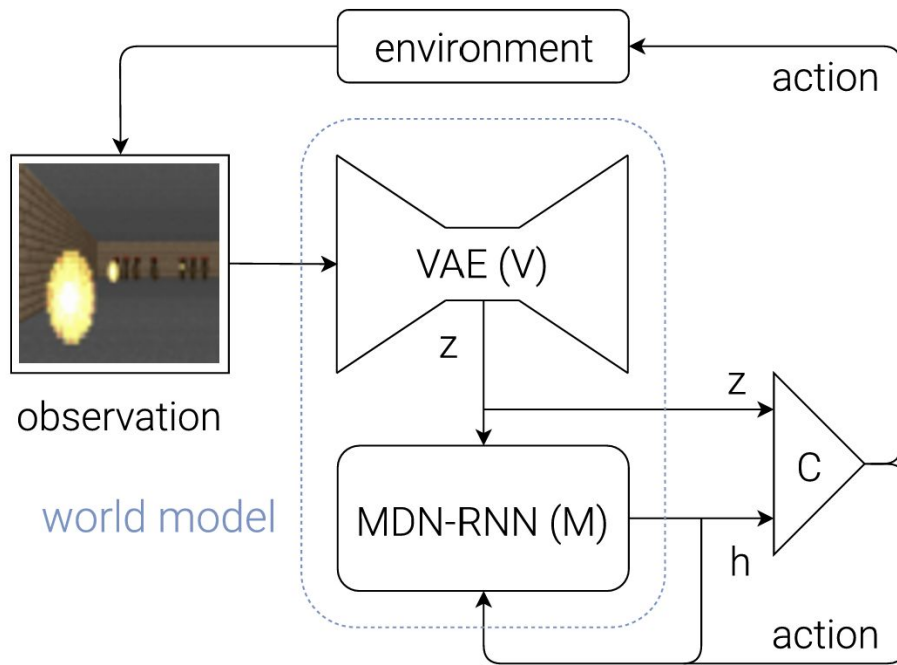
Recap: Full Model for the Agent

The raw observation is first processed by V at each time step t to produce z_t .

The input into C is this **latent vector** z_t concatenated with M 's **hidden state** h_t at each time step. (Mixture Density MDN)

C will then output an **action** vector at for **motor control**, and will affect the environment.

M will then take the **current** z_t and **action** a_t as an input to update its own hidden state to produce h_{t+1} to be used at time $t + 1$.



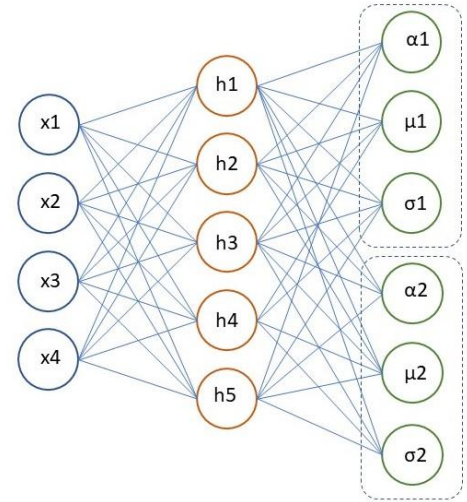
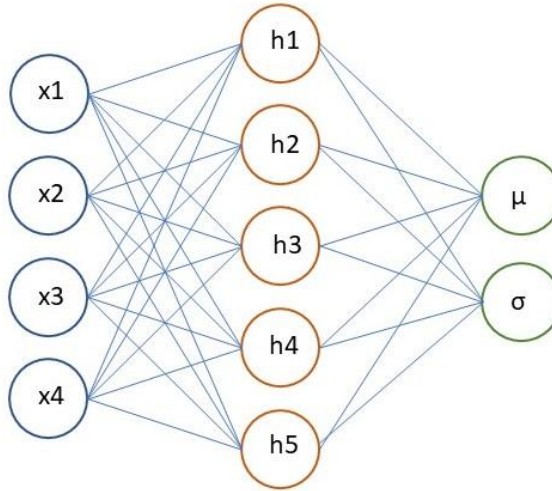
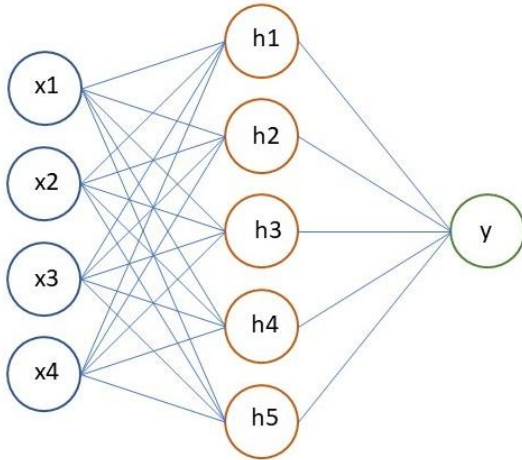
MDN

Simple NN

MDN (1 PDF)

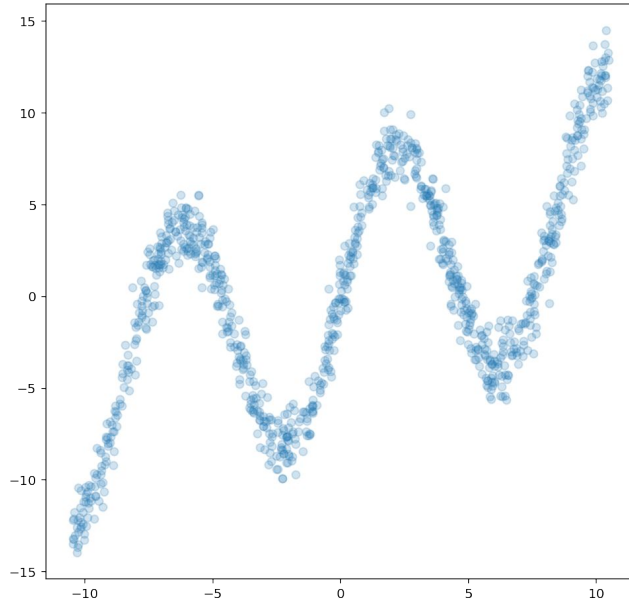
MDN (2 PDF)

GMM assumes the distribution of samples to be gaussian.

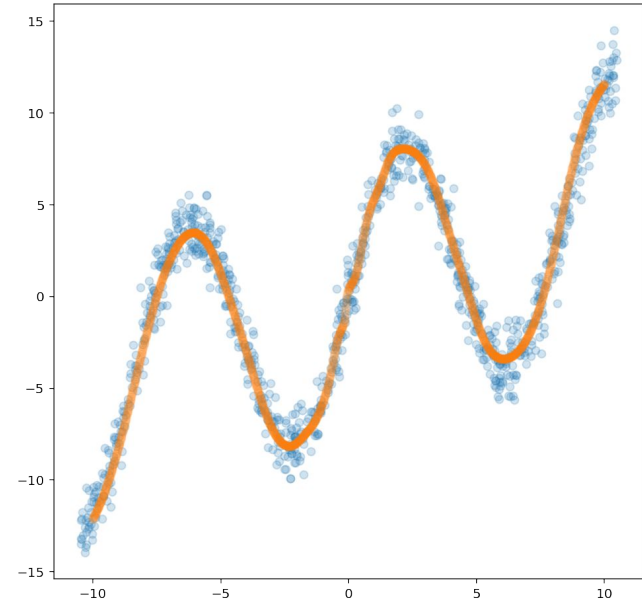


MDN

Sinusoidal Data



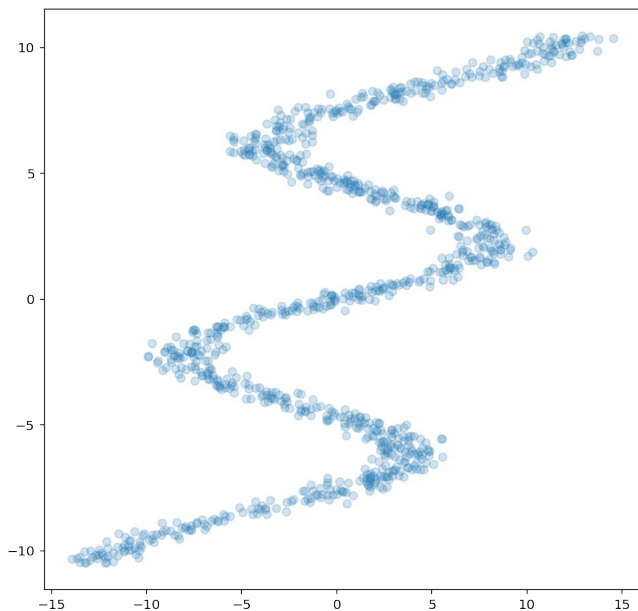
NN - Fitted Data



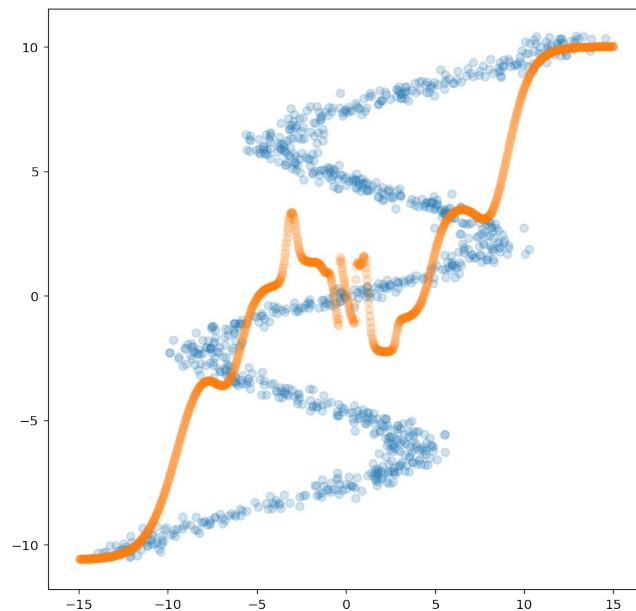
The network can fit this sinusoidal data quite well, as expected. However, this type of fitting only when we want to approximate a one-to-one, or many-to-one relationship.

MDN

Inverted Sinusoidal data



NN- Fitted Data

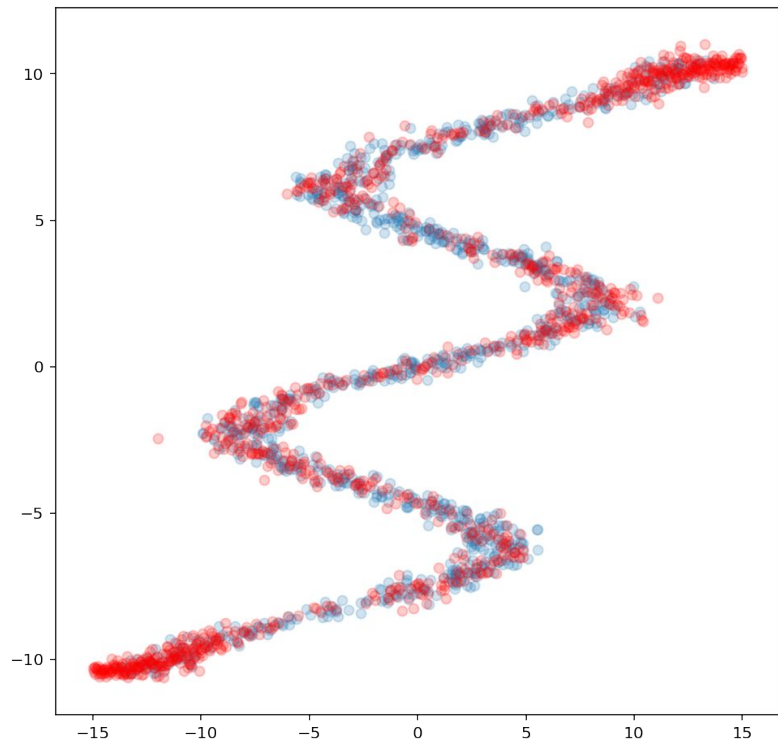


The network can't output multiple values for each input. Because using mean squared error loss, the network will try to output the average output value for each input.

MDN

A network that has the ability to predict a **multiple output values** for each **input**. Use a Mixture Density Network (MDN) to achieve this task.

- **Mixture Density Networks**, developed by Christopher Bishop in the 1990s, are one way to produce multiple outputs from a single input.
- MDN predicts a probability distribution of possible output values.
- Then can sample several possible different output values for a given input.



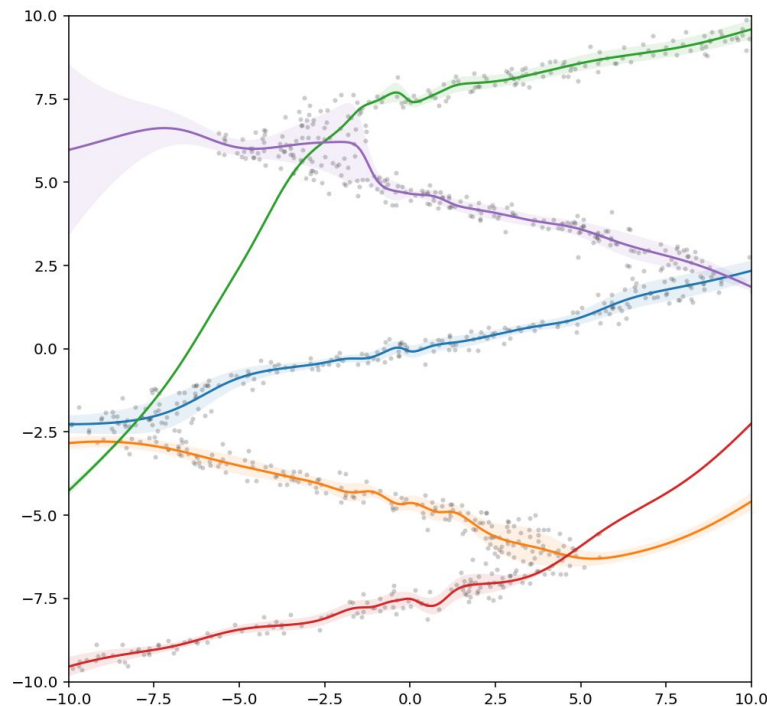
MDN

In the plot, we see that for every point on the axis, there are multiple lines or states and we select these states with probabilities modelled by the $P(z)$.

Note that the network won't find an ideal solution every time.

It's possible to get lower loss by using more Gaussians, but the results are harder to interpret.

To sample from the network pick a Gaussian k and pick a value (sample) from that Gaussian.



Sketch-RNN

It is an example of a MDN-RNN used to predict the next pen strokes of a sketch drawing.

Abstract concept of a **garden**.

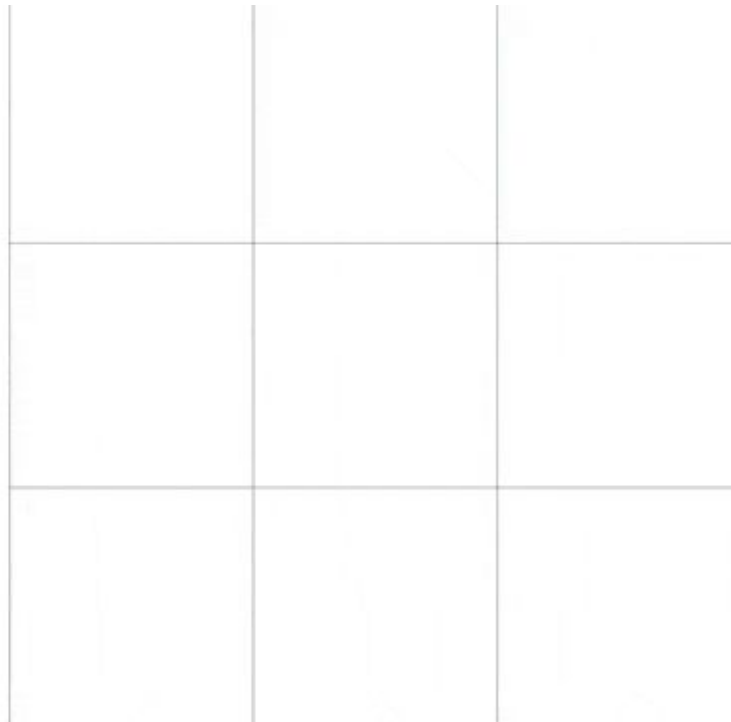
A similar model to predict the next latent vector z for Car env.



Mosquito

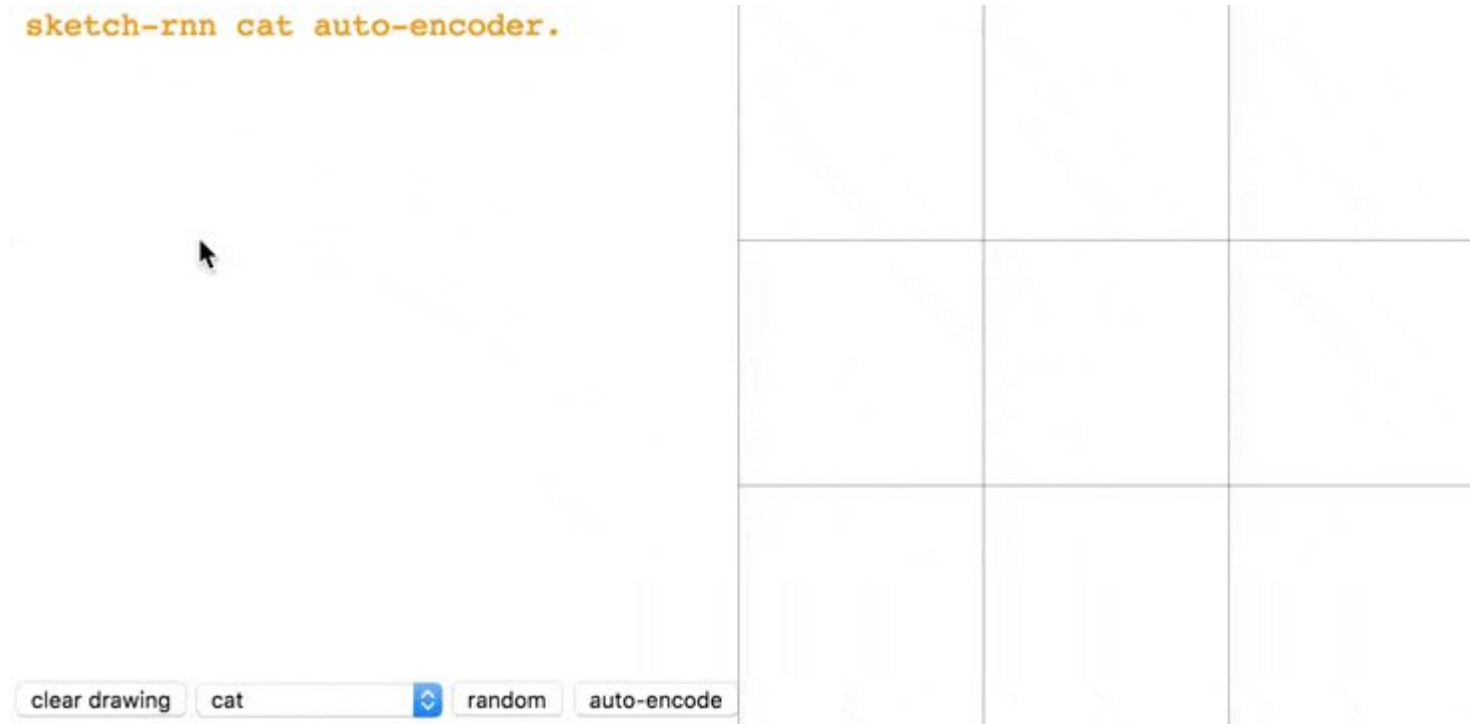
Abstract
concept of a
mosquito.

```
sketch-rnn mosquito predictor.
```



Cat

Abstract
concept of a
cat.

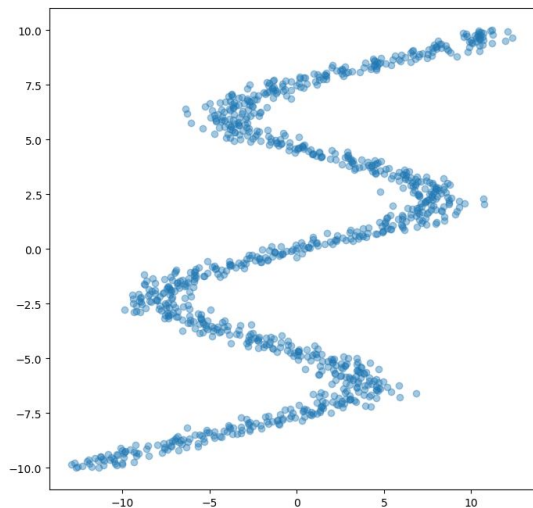


Results

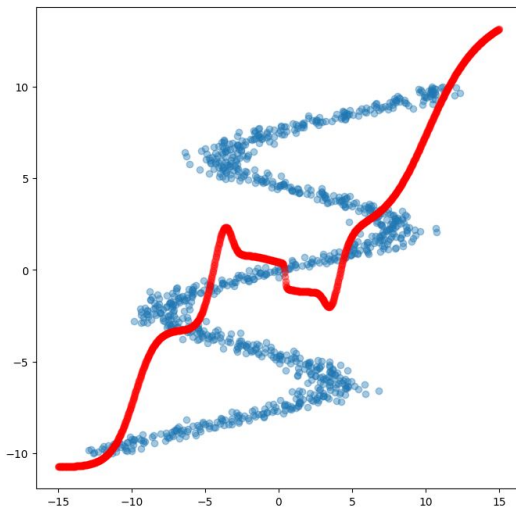
MDN (Mixture Density Network)

Data

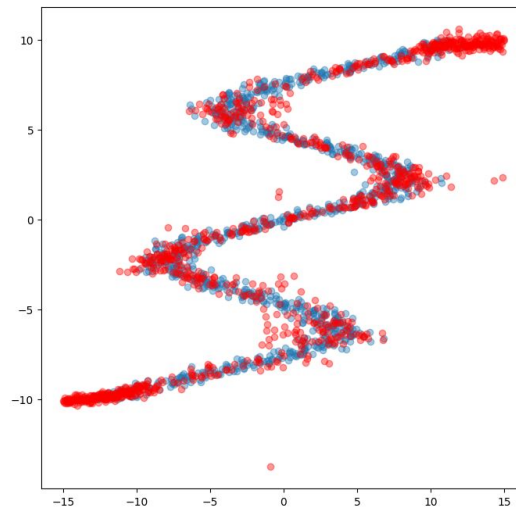
(Representing a Path)



Simple NN



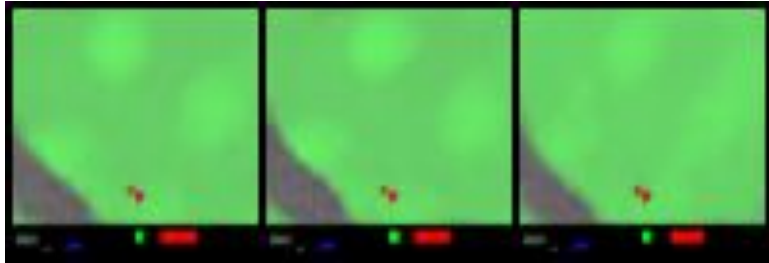
MDN Prediction



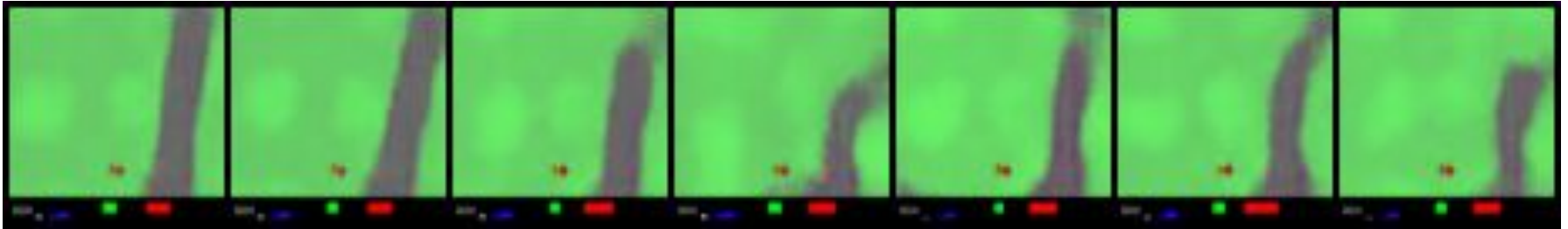
Simulation Results

RNN

Current Frame Next Frame Next Frame Prediction



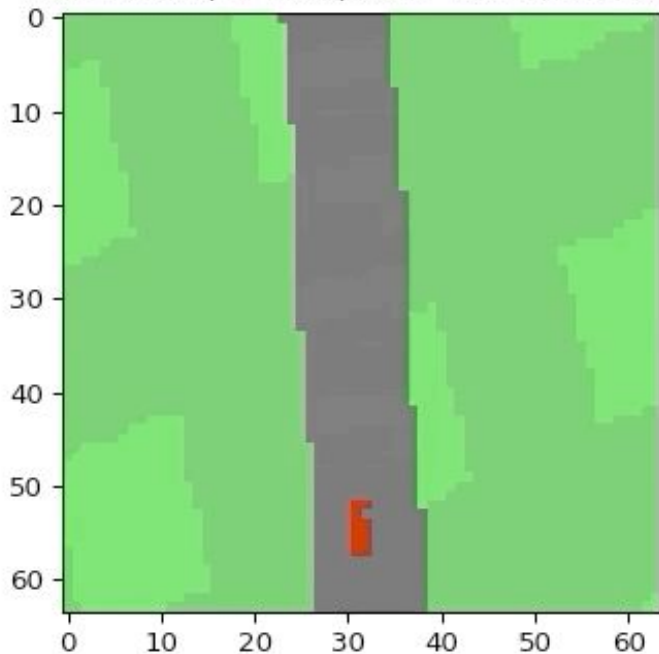
MDN -RNN



Simulation Results

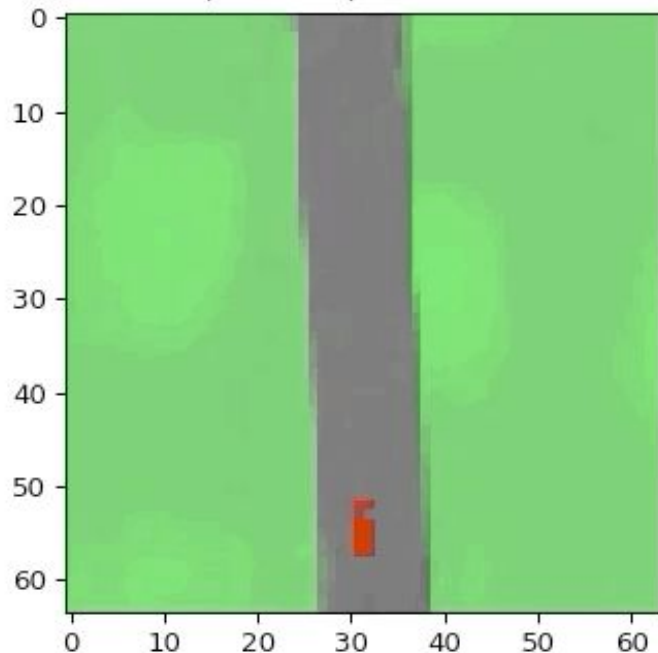
True Trajectories.

frame: 209, r: 3.40, total reward: 160.72



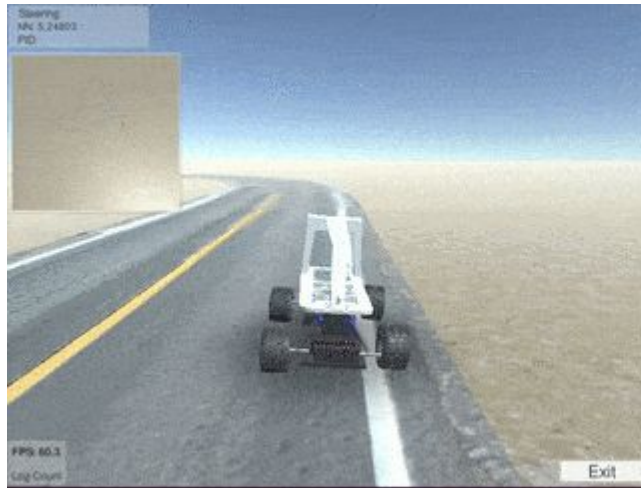
Reconstructed Trajectories.

frame: 208, r: -0.10, total reward: 157.32

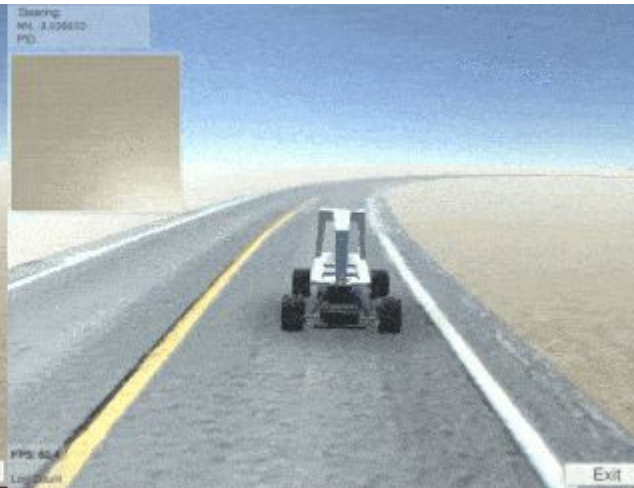


Simulator Results

Shaky navigation



Stable navigation



Simple Concepts in driving

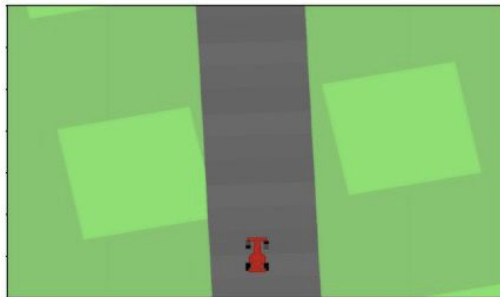
For Toy Car Model (Robot)

The car has three output action

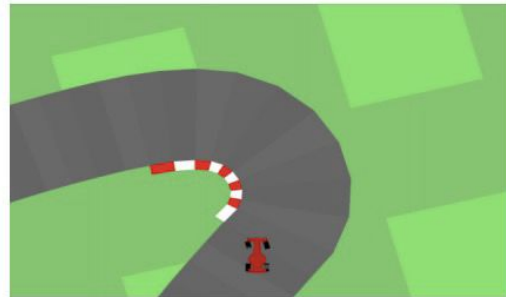
1. Steering
 - Into “turn left” and
 - “turn right”
2. Accelerating
3. Braking

Scope for Improvement

Is it possible to define abstract concepts for driving scenarios to make them “*Interpretable by design*” for deep RL.



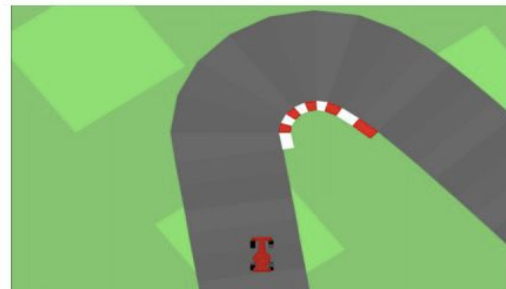
Accelerate



Brake



Turn Left



Turn Right

Defining Human-Friendly Prototypes

Future Work

1. Define human friendly prototype for realistic environment.
 - a. Vehicle dynamics (Agent)
 - i. Accelerate/ Braking
 - ii. Steering Turns
 - b. Environment dynamics
 - i. Road turns left and right.
 - ii. Lane changing / Road signs
 - iii. Obstacles (Static/Dynamic)
2. Train Prototype-Wrapper Network (PW-Net) with real world concept space.
 - a. PW-Net Explanations
 - i. Local Explanation: Explain individual action.
 - ii. Global Explanation: Explain car's (agents) overall understanding.
3. Evaluate and compare with SOTA methods.
 - a. Predict OOD failures. (Agent's crash)
 - b. Prototype comparison.